

Операционные системы

Программирование в командном процессоре ОС UNIX. Ветвления и циклы

Кулыгин Григорий Александрович

2026-04-23

Цели и задачи работы

Процесс выполнения лабораторной работы

Выводы по проделанной работе

1. Цели и задачи работы

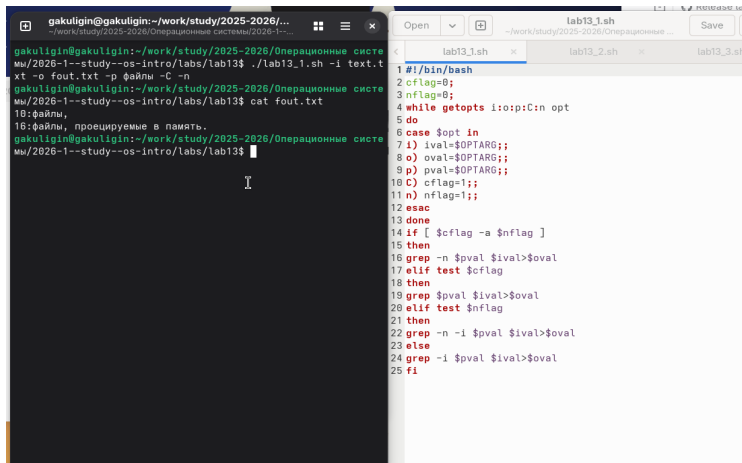
Изучить основы программирования в оболочке ОС UNIX. Научится писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

1 Выполнить 4 задания

2. Процесс выполнения лабораторной работы

1. Используя команды `getopts` `grep` напишем командный файл, который анализирует командную строку с ключами и выполним его: `-i inputfile` — прочитать данные из указанного файла; `-o outputfile` — вывести данные в указанный файл; `-p шаблон` — указать шаблон для поиска; `-C` — различать большие и малые буквы; `-n` — выдавать номера строк;

а затем ищет в указанном файле нужные строки



The image shows a terminal window on the left and a code editor on the right. The terminal window displays the execution of a shell script named `lab13_1.sh`. The user runs the script with the argument `-i text.txt`, which creates a file `fout.txt`. The user then runs `cat fout.txt`, which outputs the following text:

```
10:файлы,
16:файлы, проецируемые в память.
```

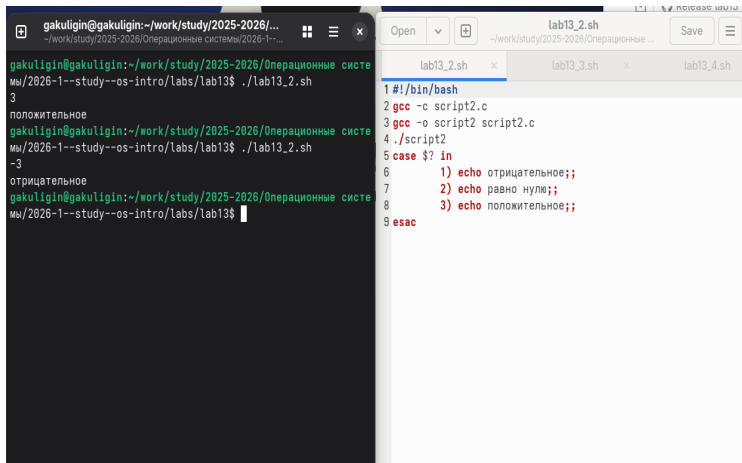
The code editor on the right shows the source code of `lab13_1.sh`. The script is a shell script that takes an input file as an argument and processes it. The code is as follows:

```
1#!/bin/bash
2cflag=0;
3nflag=0;
4while getopts i:o:p:C:n opt
5do
6case $opt in
7i) ival=$OPTARG;;
8o) oval=$OPTARG;;
9p) pval=$OPTARG;;
10C) cflag=1;;
11n) nflag=1;;
12esac
13done
14if [ $cflag -a $nflag ]
15then
16grep -n $pval $ival>$oval
17elif test $cflag
18then
19grep $pval $ival>$oval
20elif test $nflag
21then
22grep -n -i $pval $ival>$oval
23else
24grep -i $pval $ival>$oval
25fi
```

Рисунок 1: Задание 1

2. Напишем сначала на языке Си программу, которая вводит число и определяет, является ли оно больше нуля, меньше нуля или равно нулю. Затем завершим программу при помощи функции `exit(n)`, передавая информацию о коде завершения в оболочку. Командный файл вызовет эту программу и, проанализировав с помощью команды `$?`, выдаст сообщение о том, какое число было введено

Выполнение работы



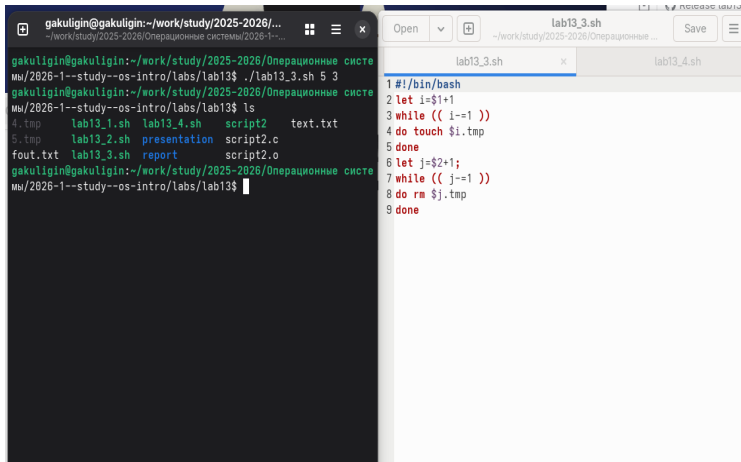
The image shows a terminal window on the left and a code editor on the right. The terminal window displays the execution of a script named `lab13_2.sh`. The script contains a `case` statement that checks the value of a variable `in` and prints different messages based on whether it is negative, zero, or positive. The code editor shows the source code of `lab13_2.sh`, which matches the script being executed in the terminal.

```
gakuligin@gakuligin:~/work/study/2025-2026/Операционные системы/2026-1--study--os-intro/labs/lab13$ ./lab13_2.sh
3
положительное
gakuligin@gakuligin:~/work/study/2025-2026/Операционные системы/2026-1--study--os-intro/labs/lab13$ ./lab13_2.sh
-3
отрицательное
gakuligin@gakuligin:~/work/study/2025-2026/Операционные системы/2026-1--study--os-intro/labs/lab13$
```

```
1 #!/bin/bash
2 gcc -c script2.c
3 gcc -o script2 script2.c
4 ./script2
5 case $? in
6     1) echo отрицательное;;
7     2) echo равно нулю;;
8     3) echo положительное;;
9 esac
```

Рисунок 2: Задание 2

3. Напишем командный файл, создающий указанное число файлов, пронумерованных последовательно от 1 до N



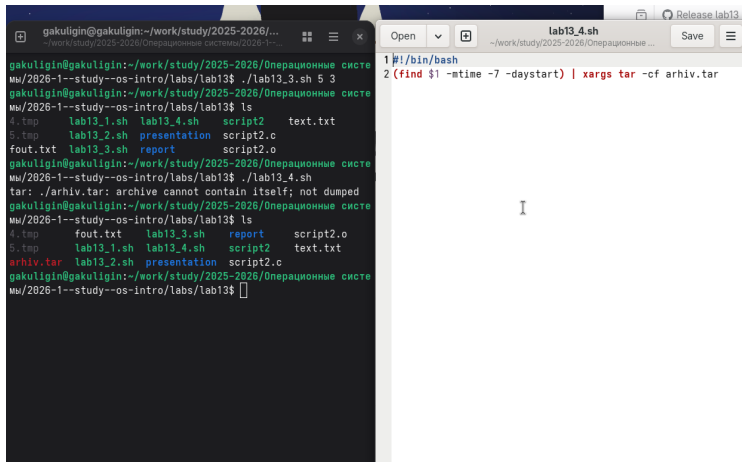
The image shows a terminal window on the left and a code editor on the right. The terminal window displays the execution of a shell script named lab13_3.sh. The user runs the script with arguments 5 and 3, then lists the contents of the directory. The code editor shows the source code of lab13_3.sh, which is a bash script that initializes a counter i to 1, enters a while loop that increments i and touches a file i.tmp until i equals 3, then increments a counter j by 2, enters another while loop that removes j.tmp until j equals 1, and finally exits.

```
gakuligin@gakuligin:~/work/study/2025-2026/Операционные системы/2026-1--study--os-intro/labs/lab13$ ./lab13_3.sh 5 3
gakuligin@gakuligin:~/work/study/2025-2026/Операционные системы/2026-1--study--os-intro/labs/lab13$ ls
4.tmp      lab13_1.sh  lab13_4.sh  script2     text.txt
5.tmp      lab13_2.sh  presentation script2.c
fout.txt   lab13_3.sh  report      script2.o
gakuligin@gakuligin:~/work/study/2025-2026/Операционные системы/2026-1--study--os-intro/labs/lab13$
```

```
1 #!/bin/bash
2 let i=$1+1
3 while (( i==1 ))
4 do touch $i.tmp
5 done
6 let j=$2+1;
7 while (( j==1 ))
8 do rm $j.tmp
9 done
```

Рисунок 3: Задание 3

4. Напишем командный файл, который с помощью команды `tar` запаковывает в архив все файлы в указанной директории. Модифицируем его так, чтобы запаковывались только те файлы, которые были изменены менее недели тому назад.



The screenshot shows a terminal window with the following content:

```
gakuligin@gakuligin:~/work/study/2025-2026/Операционные системы/2026-1--study--os-intro/labs/lab13$ ./lab13_3.sh 5 3
gakuligin@gakuligin:~/work/study/2025-2026/Операционные системы/2026-1--study--os-intro/labs/lab13$ ls
4.tmp      lab13_1.sh  lab13_4.sh  script2     text.txt
5.tmp      lab13_2.sh  presentation script2.c
fout.txt   lab13_3.sh  report      script2.o
gakuligin@gakuligin:~/work/study/2025-2026/Операционные системы/2026-1--study--os-intro/labs/lab13$ ./lab13_4.sh
tar: ./arhiv.tar: archive cannot contain itself; not dumped
gakuligin@gakuligin:~/work/study/2025-2026/Операционные системы/2026-1--study--os-intro/labs/lab13$ ls
4.tmp      fout.txt    lab13_3.sh  report      script2.o
5.tmp      lab13_1.sh  lab13_4.sh  script2     text.txt
arhiv.tar  lab13_2.sh  presentation script2.c
gakuligin@gakuligin:~/work/study/2025-2026/Операционные системы/2026-1--study--os-intro/labs/lab13$
```

On the right, a smaller window titled "lab13_4.sh" shows the script content:

```
1#!/bin/bash
2(find $1 -mtime -7 -daystart) | xargs tar -cf arhiv.tar
```

Рисунок 4: Задание 4

3. Выводы по проделанной работе

В данной работе мы изучили основы программирования в оболочке ОС UNIX и писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.